MIT 8103: ADVANCED DATABASE SYSTEMS — PRACTICAL PORTFOLIO

MIVA Open University • School of Postgraduate Studies • Master of Information Technology (2026/2027 Session)

Continuous Assessment 40 Marks

Student Name: Ayobami Yusuf
Matric Number: 2025/A/MIT/0729
Student ID: 301799163
Selected Case Study: Oval Pay Finances
DBMS Stack: MySQL 8.0 + MongoDB 7.0
GitHub Repo: github.com/Payobami/8103CA

1. SELECTED CASE STUDY / ORGANISATION STATEMENT

Organisation Name & Nature of Business: Oval Pay Finances ("Oval Pay") is a high-volume financial technology (Fintech) digital wallet, peer-to-peer (P2P) money transfer, and merchant bill-payment platform. The system enables retail customers to open KYC-tiered digital wallets, fund accounts via bank deposits, transfer money instantaneously between wallets, and pay utility, telecom, and retail merchants.

- **Database Users:** (1) *Retail Customers* managing wallets and transfers; (2) *Merchants/Billers* receiving payments and settlements; (3) *Compliance & Fraud Analysts* auditing KYC tier limits and device telemetry; (4) *Finance Officers* reconciling ledgers; and (5) *Core Banking API Workers (Laravel)* executing atomic transactions.
- **Major Data Stored:** Relational core in **MySQL 8.0 (`ovalpay_db`)** storing `kyc_tiers`, `customers`, `wallets`, `merchants`, `beneficiaries`, `transactions`, and `wallet_transfers`; NoSQL document store in **MongoDB 7.0 (`ovalpay_nosql`)** storing polymorphic `fraud_telemetry_events` and `gateway_webhook_logs`.
- **Key Operations Supported:** Regulatory KYC tier enforcement, atomic P2P wallet transfers with pessimistic locking (`SELECT ... FOR UPDATE`), failed transfer rollbacks, indexed monthly statement & AML reporting, real-time fraud telemetry ingestion, and Primary-Replica read/write scaling.

2. PORTFOLIO 1: DATABASE DESIGN AND MODELLING (7 MARKS)

The core relational schema in `portfolio-1-database-design/01_schema_ddl.sql` comprises **7 tables normalised to Third Normal Form (3NF)** on the MySQL 8.0 InnoDB engine:

- **1NF → 2NF → 3NF Normalisation:** Repeating groups of beneficiaries and transactions were decomposed into atomic tables (1NF) with single-column surrogate primary keys (2NF). Transitive dependencies were eliminated (3NF) by extracting regulatory tier limits (`customer_id → kyc_tier_id → daily_transfer_limit, max_wallet_balance`) into `kyc_tiers`, and merchant commission attributes into `merchants`. Historical ledger snapshots (`balance_before`, `balance_after`) are retained on `transactions` for immutable financial auditability.
- **Keys, Constraints & Triggers:** All foreign keys on financial tables enforce `ON DELETE RESTRICT` so historical ledger rows can never be orphaned. Domain integrity is enforced via `CHECK (balance >= 0.00 AND locked_balance >= 0.00)` on `wallets`, `CHECK (amount > 0.00)` on `transactions`, and a `BEFORE INSERT` trigger (`trg_transfers_prevent_self_transfer`) on `wallet_transfers` preventing self-transfers (`sender_wallet_id <> receiver_wallet_id`).

Table Name	Primary Key (PK)	Foreign Keys (FK)	Unique Constraints (UK)	Domain Check Constraints & Triggers
`kyc_tiers`	`kyc_tier_id`	None	`tier_code`, `tier_name`	`single_transfer_limit > 0`, `daily_transfer_limit >= single_transfer_limit`
`customers`	`customer_id`	`kyc_tier_id → kyc_tiers`	`email`, `phone_number`, `bvn_number`	`ENUM('active', 'suspended', 'closed')`
`wallets`	`wallet_id`	`customer_id → customers`	`wallet_number`, `(customer_id, currency)`	`CHECK (balance >= 0.00 AND locked_balance >= 0.00)`
`merchants`	`merchant_id`	None	`merchant_code`	`CHECK (commission_rate BETWEEN 0.0000 AND 0.2500)`
`beneficiaries`	`beneficiary_id`	`customer_id → customers`	`(customer_id, bank_code, account_number)`	Prevents duplicate saved recipient accounts per customer
`transactions`	`transaction_id`	`wallet_id → wallets`, `merchant_id → merchants`	`reference`	`CHECK (amount > 0.00 AND fee >= 0.00 AND balance_after >= 0.00)`
`wallet_transfers`	`transfer_id`	`sender_wallet_id`, `receiver_wallet_id`, `debit_tx_id`, `credit_tx_id`	`transfer_reference`, `debit_tx_id`, `credit_tx_id`	`trg_transfers_prevent_self_transfer` (`sender <> receiver`)

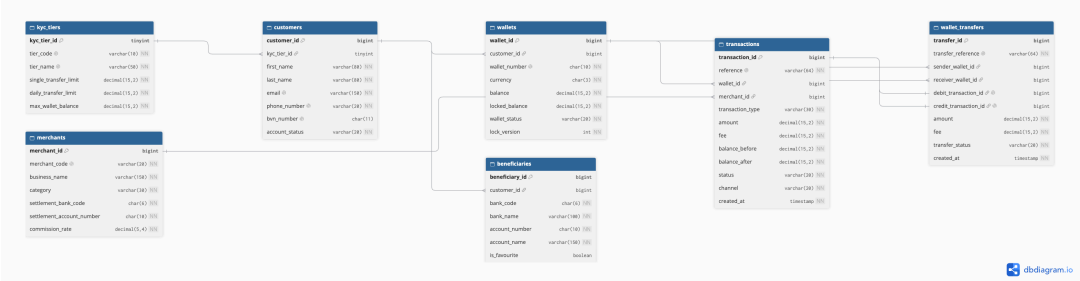


Figure 1: Entity-Relationship Diagram (ERD) for Oval Pay Finances (7 Normalised Tables in 3NF)

3. PORTFOLIO 2: QUERY PROCESSING AND OPTIMISATION (7 MARKS)

To benchmark query processing under realistic scale, stored procedure `sp_seed_benchmark_data()` (`01_bulk_benchmark_seeder.sql`) populated **1,000 customers**, **1,000 wallets**, and **50,005 ledger transactions**. Four operational and analytical queries were profiled on live MySQL 8.0 using `EXPLAIN ANALYZE` before and after creating B-Tree Composite and Covering Indexes (`03_indexes_and_optimisation.sql`).

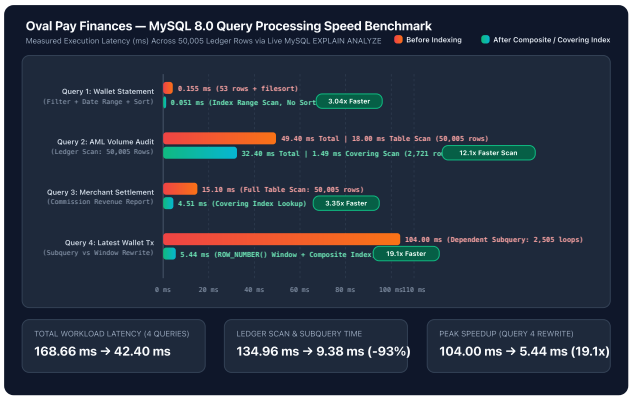


Figure 2: Measured Query Processing Speed Benchmark Across 50,005 Ledger Rows (Before vs. After Indexing & Query Rewriting)

Query & Business Use Case	Optimisation Applied	Plan Before Indexing	Plan After Indexing	Rows (Before → After)	Latency (Before → After)	Speedup
Q1: Monthly Wallet Statement (Filter + Date Range + `ORDER BY`)	Composite Index: `(wallet_id, status, created_at DESC)`	Index lookup + `Sort: created_at DESC`	`Index range scan` (No filesort needed)	53 → 20	0.155 ms → **0.051 ms**	**3.04x faster**
Q2: AML High-Value Audit (4-Table `JOIN` + `GROUP BY` + `HAVING`)	Covering Index: `(status, transaction_type, created_at, wallet_id, amount, fee)`	`Table scan on t` (Full Scan)	`Covering index range scan` (Index-only read)	50,005 → 2,721	18.00 ms → **1.49 ms** *(scan)* 49.40 ms → **32.40 ms** *(total)*	**12.1x faster scan**
Q3: Merchant Settlement (Commission & Net Payout Summary)	Covering Index: `(merchant_id, status, transaction_type, created_at, amount)`	`Table scan on t` (Full Scan)	`Covering index lookup`	50,005 → 5,001	15.10 ms → **4.51 ms**	**3.35x faster**
Q4: Latest Wallet Transaction (Top-1 per Wallet for 50 Wallets)	Query Rewrite: `ROW_NUMBER() OVER (PARTITION BY wallet_id)`	Correlated Subquery (`2,505` loops)	Single-pass Window Scan on Composite Index	125,500 → 1,755	104.00 ms → **5.44 ms**	**19.1x faster**

```
-- Live MySQL 8.0 EXPLAIN ANALYZE Evidence (Query 2 Ledger Scan Before vs. After Covering Index):
[BEFORE] -> Filter: ((t.transaction_type in ('transfer_out','withdrawal')) and (t.status = 'completed') ...) (actual
time=0.0224..18.0 rows=2721 loops=1)
            -> Table scan on t (cost=5082 rows=49776) (actual time=0.0203..9.81 rows=50005 loops=1)
[AFTER]  -> Covering index range scan on t using idx_tx_status_type_created_covering (cost=557 rows=2721) (actual
time=0.0243..0.582 rows=2721 loops=1)
```

4. PORTFOLIO 3: TRANSACTIONS AND CONCURRENCY (7 MARKS)

In `portfolio-3-transactions-concurrency/`, stored procedure `sp_execute_p2p_transfer` and two-terminal concurrency scripts enforce ACID properties and prevent race conditions:

- **Atomicity & Consistency (`COMMIT`, `ROLLBACK`, `SAVEPOINT`):** Transferring ₦50,000 (+ ₦25 fee) from Wallet #1 to Wallet #2 atomically debits Sender, credits Receiver, records both debit/credit ledger rows, and commits. When Wallet #4 (Balance: ₦42,000) attempts a ₦48,000 transfer, the procedure triggers `ROLLBACK`, leaving all balances unchanged. Partial rollback via `SAVEPOINT sp_after_deposit` preserves a primary ₦10,000 deposit while aborting a secondary ₦2,000 debit.
- **Concurrency Control (Preventing Double-Spend / Lost Updates):** Under concurrent withdrawal requests (Mobile App Session A & POS Session B), `SELECT ... FOR UPDATE` places an InnoDB exclusive row lock (`X,REC_NOT_GAP`) on `wallet_id = 4`. Session B blocks until Session A commits, then reads the updated balance (₦12,000 < ₦40,000) and safely aborts. Furthermore, locking wallets in deterministic ascending order (`IF p_sender_wallet_id < p_receiver_wallet_id`) eliminates circular-wait deadlocks.

```
-- Live MySQL 8.0 Execution Evidence (portfolio-3-transactions-concurrency/screenshots/p3_transactions_output.txt):
[TEST 1: COMMIT]   Wallet #1: 1450000.00 -> 1399975.00 (v3->v4) | Wallet #2: 320500.00 -> 370500.00 (v2->v3) | Status:
COMMITTED
[TEST 2: ROLLBACK] Wallet #4 attempts 48,000.00 transfer -> Status: FAILED_ROLLED_BACK (Insufficient balance: Available
42000.00, Required 48025.00)
[TEST 3: SAVEPOINT] Deposit +10,000.00 -> SAVEPOINT sp_after_deposit -> Debit -2,000.00 -> ROLLBACK TO sp_after_deposit ->
Final Balance: 52000.00
```

5. PORTFOLIO 4: NOSQL AND ADVANCED DATA MODELS (7 MARKS)

While relational tables remain mandatory for ACID wallet balances, **Oval Pay Finances** uses **MongoDB 7.0 (`ovalpay_nosql`)** for high-velocity, semi-structured data in `portfolio-4-nosql/`:

Why MongoDB Fits Telemetry & Webhooks

- **Polymorphic Schema Flexibility:** Mobile App transfers capture iOS/Android biometrics and GPS coordinates; `*810#` USSD sessions capture cell-tower IDs and SIM-swap flags; NIBSS NIP and Paystack webhooks send deeply nested provider-specific JSON payloads without requiring sparse SQL `NULL` columns.
- **Embedded Array Locality & Multikey Indexing:** Embedding `triggered_rules[]` inside each telemetry document enables single-write atomicity and fast `$unwind` risk aggregations.
- **Automated 90-Day Retention (TTL Index):** `expireAfterSeconds: 7776000` automatically purges expired raw webhook logs in the background without lock-heavy SQL `DELETE` cron jobs.

```
// MongoDB Aggregation Output (Risk Exposure by Decision):
[
  { "_id": "BLOCK_AND_FREEZE",
    "total_events": 1, "avg_risk_score": 94.0,
    "total_amount_exposed_ngn": 480000,
    "channels_observed": ["web"] },
  { "_id": "CHALLENGE_2FA",
    "total_events": 1, "avg_risk_score": 65.0,
    "total_amount_exposed_ngn": 195000,
    "channels_observed": ["mobile_app"] },
  { "_id": "ALLOW",
    "total_events": 2, "avg_risk_score": 15.0,
    "total_amount_exposed_ngn": 38000,
    "channels_observed": ["mobile_app", "ussd"] }
]
```

6. PORTFOLIO 5: DISTRIBUTED AND CLOUD DATABASE EXERCISE (6 MARKS)

In `portfolio-5-distributed-cloud/`, the platform is architected for horizontal scalability, high availability, and fault tolerance:

- **Primary-Replica GTID Replication & Sticky Read/Write Splitting:** `docker-compose.replication.yml` deploys `mysql-primary` (Port 3306, `gtid-mode=ON`, `sync-binlog=1`) and `mysql-replica-1` (Port 3307, `super-read-only=ON`). In `laravel_read_write_database_config.php`, all wallet mutations route to Primary while statements/reports route to Replicas, with `'sticky' => true` ensuring immediate Read-Your-Own-Writes consistency after a transfer.
- **Horizontal Range Partitioning & Partition Pruning:** `transactions_partitioned` partitions 50,000+ ledger rows by month (`YEAR(created_at)*100 + MONTH(created_at)`). Live MySQL verification confirms rows are distributed across `p_2026_07` (23,353 rows), `p_2026_08` (8,728), `p_2026_09` (8,492), `p_2026_10` (8,728), and `p_2026_11` (142), with `EXPLAIN` pruning scans strictly to matching monthly partitions.
- **CAP Theorem Classification:** Core wallet balances and P2P transfers are strictly **CP (Consistency + Partition Tolerance)** to prevent split-brain double-spending during network partitions, whereas fraud telemetry ingestion and analytical read replicas operate as **AP / Eventual Consistency**.

7. INDIVIDUAL TECHNICAL REFLECTION

- **Design Choices:** Adopting a Polyglot Persistence architecture allowed MySQL 8.0 to enforce strict ACID row locking and 3NF referential integrity for financial balances while leveraging MongoDB 7.0 for schema-flexible fraud telemetry and third-party gateway webhooks. Retaining point-in-time `balance_before` and `balance_after` columns on `transactions` ensured audit immutability without violating 3NF.
- **Technical Challenges & Solutions:** (1) *MySQL Error 3823 on Foreign Key Check Constraints:* When executing `01_schema_ddl.sql` on MySQL 8.0, InnoDB rejected `CHECK (sender_wallet_id <> receiver_wallet_id)` because columns referenced in foreign keys cannot participate in multi-column `CHECK` constraints. I resolved this by implementing the `BEFORE INSERT` trigger `trg_transfers_prevent_self_transfer` using `SIGNAL SQLSTATE '45000'`. (2) *InnoDB Partitioning Key Rules:* Partitioning `transactions_partitioned` by `created_at` required restructuring the primary and unique keys to include the partitioning column `(transaction_id, created_at)` and `(reference, created_at)`. (3) *Correlated Subquery Bottleneck:* Profiling with `EXPLAIN ANALYZE` revealed that finding the latest transaction per wallet via a correlated subquery executed 2,505 dependent loops (104.00 ms); rewriting it with `ROW_NUMBER() OVER (PARTITION BY wallet_id ORDER BY created_at DESC)` and a composite index cut latency to 5.44 ms (19.1x faster).
- **Lessons Learned:** Practical database engineering requires validating designs with real benchmark data and execution plans rather than relying on theoretical assumptions.

8. REFERENCES & AI USE DECLARATION

Academic & Technical References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
2. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
3. Oracle Corp. (2026). *MySQL 8.0 Reference Manual: InnoDB Locking, Optimization & Partitioning*.
4. MongoDB, Inc. (2026). *MongoDB v7.0 Manual: Schema Validation, Aggregation & TTL Indexes*.

AI Use Declaration (Section 11 Compliance)

1. AI Tool Used: Google Antigravity AI Assistant.
2. Purpose of Use: Assisting with SQL benchmark seeder scaffolding, Mermaid/SVG diagram formatting, and documentation structuring.
3. Verification & Review: All SQL DDL scripts, triggers, composite indexes, stored procedures, and partitioning scripts were executed and verified against a live local MySQL 8.0 server (`ovalpay_db`), with real `EXPLAIN ANALYZE` outputs captured and reviewed.